

Phần 3 - Architectural Thinking

Mục lục

3.1 Tổng quan Phần 3: Architectural Thinking	3
Nếu bạn đến từ Data Science	3
Vì sao có phần này	3
Phần 3 sẽ dạy gì	3
Bài 3.2: Architecture vs Design	3
Bài 3.3: Trade-off Analysis	3
Bài 3.4: Modularity	4
Thứ tự đọc	4
Kết nối với phần sau	4
Mindset shift sau Phần 3	4
3.2 Architecture vs Design	5
Nếu bạn đến từ Data Science	5
Câu hỏi mở đầu	5
Ba góc nhìn về architecture vs design	5
Góc nhìn 1: Theo scope	5
Góc nhìn 2: Theo cost-of-change	7
Góc nhìn 3: Theo reversibility (Bezos)	7
Ranh giới mềm theo context	7
Ví dụ: Chọn ORM	7
Ví dụ: Class hierarchy của domain Order	7
Levels of Knowledge (Knowledge Triangle)	7
1. Stuff you know (Bạn biết)	7
2. Stuff you know you don't know (Bạn biết là bạn chưa biết)	9
3. Stuff you don't know you don't know (Bạn không biết là mình không biết)	9
Vì sao quan trọng cho architect	9
Cách mở rộng vùng 2	9
"Architecture is the stuff that's hard to Google"	9
Architect cũng phải code	9
Sai lầm thường gặp	10
Sai lầm 1: Coi mọi diagram là architecture	10
Sai lầm 2: Tranh cãi mọi quyết định ở level architectural	10
Sai lầm 3: Bỏ qua context khi học pattern	10
Sai lầm 4: Quá xa rời code	10
Tóm tắt	10
3.3 Phân tích Trade-off	11

Câu hỏi nền tảng	11
Nếu bạn đến từ Data Science	11
Vì sao trade-off khó	11
Lý do 1: Lợi ích thường visible, chi phí thì không	11
Lý do 2: Chi phí phân tán theo thời gian	12
Lý do 3: Stakeholder khác nhau ưu tiên khác nhau	12
Lý do 4: Khó định lượng	12
Framework ATAM-lite	12
Bước 1: Liệt kê options (3-5 cái)	12
Bước 2: Liệt kê quality attributes liên quan	12
Bước 3: Identify trade-off	13
Bước 4: Map vào priorities	13
Bước 5: Document quyết định	13
Sai lầm “tối ưu một chiều”	14
Ví dụ thật	14
Cách phòng tránh	14
Cân bằng Architect và Hands-on Coding	14
Hai cực sai	14
Sweet spot	15
Dấu hiệu architect đã xa rời code	15
Trade-off đặc biệt: “Right-sizing” architecture	15
Quy tắc thực hành: “Architecture should match team size + 1 stage”	15
Sai lầm thường gặp	16
Sai lầm 1: “Hype-driven architecture”	16
Sai lầm 2: “Resume-driven architecture”	16
Sai lầm 3: “Last project bias”	16
Sai lầm 4: “Default to complex”	16
Tóm tắt	16
3.4 Modularity	17
Vì sao quan trọng	17
Big Ball of Mud	17
Distributed Monolith	17
Nếu bạn đến từ Data Science	17
Nguyên tắc: High cohesion at module boundary	18
Vertical vs Horizontal Slicing	18
Horizontal Slicing (theo layer)	18
Vertical Slicing (theo feature/domain)	18
Khi nào dùng cái nào	19
Bounded Context (từ DDD)	19
Anti-patterns chi tiết	20
Anti-pattern 1: Big Ball of Mud (BBoM)	20
Anti-pattern 2: Distributed Monolith	20
Anti-pattern 3: Death Star	20
Modularity ở các context khác nhau	20
Trong monolith	20
Trong microservices	21
Trong mobile apps	21
Trong frontend SPA	21

Heuristic identify module boundary	21
Heuristic 1: Theo “Two-Pizza Team” của Amazon	22
Heuristic 2: Theo change rate	22
Heuristic 3: Theo data ownership	22
Heuristic 4: Theo bounded context	22
Cẩn thận: Modularity có cost	22
Tóm tắt	22
Tổng kết Phần 3	22

3.1 Tổng quan Phần 3: Architectural Thinking

Tóm tắt một dòng: Phần 2 dạy bạn viết code tốt ở mức class/module. Phần 3 dạy bạn *suy nghĩ như architect*, phân biệt cái gì là architectural, cân nhắc trade-off có hệ thống, và scale principles modularity từ module lên hệ thống.

Nếu bạn đến từ Data Science

Phần 3 là bước chuyển từ tư duy “chọn model nào” sang “chọn hình dạng hệ thống nào”. Một Data Scientist đã quen so sánh model bằng metric. Architect cũng so sánh option, nhưng metric là latency, cost, freshness, privacy, reliability và maintainability. Đây là phần giúp bạn học cách lập luận khi không có một đáp án đúng tuyệt đối.

Vì sao có phần này

Bạn vừa hoàn thành Phần 2, master SOLID. Có thể bạn nghĩ “vậy là biết kiến trúc rồi”. Sai. SOLID là *necessary* (cần) nhưng không *sufficient* (đủ) cho kiến trúc tốt. Một codebase có thể tuân thủ 100% SOLID nhưng vẫn vỡ ở mức kiến trúc vì:

1. **Chọn sai quy mô.** Microservices cho startup 5 người (over-engineer) hoặc monolith cho hệ enterprise 200 engineers (bottleneck).
2. **Không cân nhắc trade-off.** Tối ưu performance hy sinh maintainability mà không nhận ra.
3. **Boundary sai giữa các module/service.** Cohesion thấp ở mức hệ thống dù mỗi module nội bộ có cohesion cao.
4. **Document không có.** Kiến trúc tồn tại trong đầu 1 người, evolve không kiểm soát.

Phần 3 fix tư duy về 3 vấn đề đầu (vấn đề 4 thuộc Phần 7).

Phần 3 sẽ dạy gì

Bốn bài:

Bài 3.2: Architecture vs Design

Phân biệt quyết định nào là architectural (đắt khi sửa, ảnh hưởng QA), quyết định nào là design thuần (dễ sửa, scope module). Đưa framework 3 tiêu chí từ Bài 1.2 vào thực hành: làm bài tập nhận diện trên cases thật. Quan trọng: trình bày “Levels of Knowledge”, phân loại kiến thức của technologist thành 3 mức (stuff you know, stuff you know you don’t know, stuff you don’t know you don’t know), giải thích vì sao architect cần đầu tư vào mức thứ hai.

Bài 3.3: Trade-off Analysis

Mọi quyết định kiến trúc đều có trade-off. Không có “best architecture”, chỉ có “best for current context”. Bài này dạy:

- Framework ATAM-lite để liệt kê trade-off một cách có hệ thống.
- Sai lầm phổ biến: chỉ đánh giá theo 1 chiều (vd: chỉ tối ưu performance, bỏ qua maintainability).
- Cách present trade-off cho stakeholder không-kỹ-thuật (PM, business).
- “Architecture is the stuff you can’t Google”, quote nổi tiếng giải thích vì sao trade-off là essence của SA.

Bài 3.4: Modularity

Scale cohesion-coupling từ class (Bài 2.2) lên module/sub-system/service. Bài này đi qua:

- Tiêu chí nhận diện module boundary đúng (bounded context từ DDD ở mức đơn giản).
- Sai lầm: “horizontal slicing” (chia theo layer: UI/Service/Repo) vs “vertical slicing” (chia theo feature/domain), và vì sao vertical thường tốt hơn ở mức kiến trúc.
- Pattern thường gặp: deathstar antipattern, big ball of mud, distributed monolith.
- Áp dụng cho monolith (chia thành modules), cho microservices (chia thành services), cho mobile (chia thành features).

Thứ tự đọc

Đề nghị đọc tuần tự 3.2 □ 3.3 □ 3.4 vì:

- 3.3 (trade-off) dùng concept “architectural decision” từ 3.2.
- 3.4 (modularity) áp dụng cả 3.2 và 3.3 vào việc chia module.

Mỗi bài 3000-4500 chữ, đọc 1.5-2h.

Kết nối với phần sau

- **Phần 4 (Quality Attributes)** bổ sung *cái cần tối ưu* cho trade-off analysis ở 3.3.
- **Phần 5-6 (Architecture Styles)** áp khái niệm modularity ở 3.4 vào các style cụ thể (mỗi style là một cách chia module).
- **Phần 7 (Documenting)** tài liệu hoá kết quả của 3.2-3.3 thông qua ADR.
- **Phần 8 (Case Studies)** áp toàn bộ tư duy phần 3 vào dự án thực.

Mindset shift sau Phần 3

Sau khi đọc xong phần này, bạn sẽ:

1. Đọc một PR/design proposal và phân biệt được câu hỏi nào là kiến trúc (đáng cân nhắc kỹ) và câu hỏi nào là implementation detail (để dev tự quyết).
2. Khi gặp xung đột về design giữa 2 team, không vội “chọn 1 bên” mà liệt kê trade-off để team chọn theo priority business.
3. Khi vẽ system diagram, biết cách chia module theo domain (vertical) thay vì layer (horizontal).

Vào [Bài 3.2: Architecture vs Design](#) để bắt đầu.

3.2 Architecture vs Design

Tóm tắt một dòng: Không có ranh giới cứng giữa architecture và design, ranh giới phụ thuộc context. Heuristic thực hành: hỏi 3 câu (bao trùm? ảnh hưởng QA? đắt khi sửa?) cho mỗi quyết định để biết nó deserve cấp attention nào.

Nếu bạn đến từ Data Science

Trong ML, không phải mọi quyết định đều là kiến trúc. Chọn learning rate, số epoch hoặc threshold ban đầu thường là design/experiment decision. Nhưng chọn batch inference hay online inference, chọn feature store hay query trực tiếp warehouse, chọn lưu model artifact ở registry nào, chọn Kafka hay REST cho prediction events, đó là architectural decision vì ảnh hưởng toàn hệ thống và khó đổi sau này.

Câu hỏi mở đầu

Hãy xét 5 quyết định sau và đoán xem cái nào là architectural:

1. Dùng React hay Vue cho frontend của một SaaS B2B.
2. Đặt tên endpoint là `/api/users` hay `/api/v1/users`.
3. Lưu password bằng bcrypt hay Argon2.
4. Đặt timeout cho HTTP client là 5s hay 10s.
5. Chia order processing thành 1 service hay 3 service.

Suy nghĩ trước khi đọc tiếp.

...

Câu trả lời theo framework 3 tiêu chí (từ Bài 1.2: bao trùm toàn hệ, ảnh hưởng QA, đắt khi sửa):

1. **React vs Vue:** Architectural. Cả 3 tiêu chí thoả mãn (bao trùm frontend, ảnh hưởng productivity/performance, đổi sau 2 năm tốn 6-12 tháng).
2. **/users vs /v1/users:** Borderline architectural. Tiêu chí 1 và 3 thoả (mọi client phải tuân, đổi sau khó), tiêu chí 2 thì không mạnh. Vẫn cần thảo luận team trước khi chốt.
3. **bcrypt vs Argon2:** Design decision. Bao trùm (đúng) nhưng đổi sau dễ (migrate hashing có pattern chuẩn), ảnh hưởng QA không lớn (cả hai đều secure đủ).
4. **Timeout 5s vs 10s:** Implementation detail. Đổi runtime qua config.
5. **1 vs 3 services:** Architectural (mạnh). Cả 3 tiêu chí mạnh.

Phân loại đúng giúp bạn:

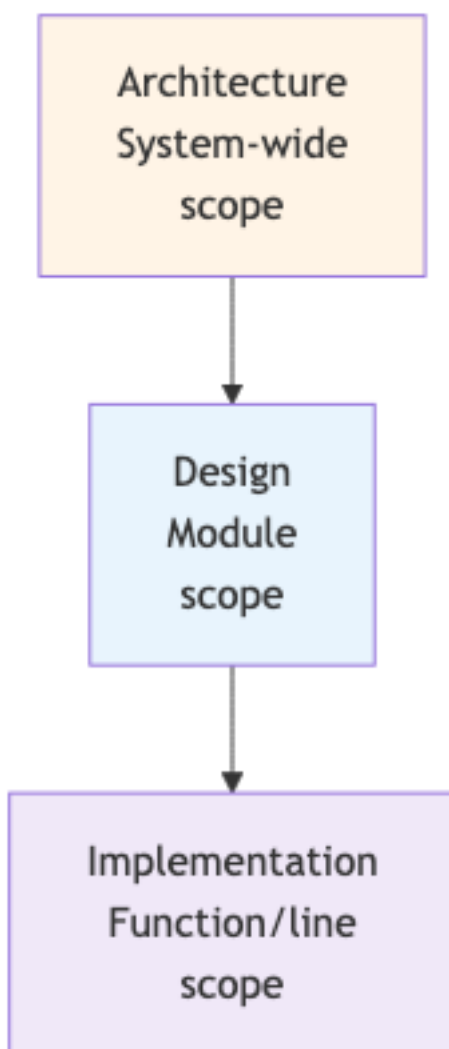
- Dành thời gian thảo luận đúng chỗ (architectural cần meeting; implementation detail không).
- Document đúng nơi (architectural vào ADR; design vào code comment).
- Empower team đúng cách (architectural cần consensus; implementation detail để dev tự quyết).

Ba góc nhìn về architecture vs design

Góc nhìn 1: Theo scope

- **Architecture:** structure của toàn hệ, services, communication, data flow.
- **Design:** structure bên trong một module, class, interface, pattern.
- **Implementation:** cấu trúc bên trong một function, algorithm, naming, control flow.

Mỗi cấp depend cấp trên. Architecture định nghĩa boundary; design điền vào chi tiết bên trong; implementation realise.



Hình 1: Sơ đồ 6

Góc nhìn 2: Theo cost-of-change

Cấp	Time-to-change	Example
Implementation	Minutes	Rename variable, fix typo
Design	Hours	Refactor class hierarchy trong module
Architecture	Months	Migrate database, split service

Cost-of-change tăng theo cấp. Architectural decisions đáng deserve thời gian cân nhắc tỷ lệ với cost.

Góc nhìn 3: Theo reversibility (Bezos)

Jeff Bezos chia decisions thành 2 loại:

- **Type 1 (one-way doors):** khó/không reverse. Phải cân nhắc kỹ.
- **Type 2 (two-way doors):** dễ reverse. Quyết nhanh, thử, sửa.

Architecture thường là Type 1. Design và implementation thường là Type 2.

Heuristic mạnh: **Type 1 decisions cần meeting + ADR. Type 2 decisions cho 1 người quyết và move on.**

Ranh giới mềm theo context

Quan trọng: cùng quyết định có thể là architectural trong context A nhưng design trong context B.

Ví dụ: Chọn ORM

- **Startup 3 người, MVP 6 tháng:** Implementation choice. Đổi sau dễ vì code base nhỏ.
- **Enterprise 50 services dùng cùng ORM 5 năm:** Architectural. Đổi = dự án 12 tháng.

Ví dụ: Class hierarchy của domain Order

- **App đơn giản với 1 loại Order:** Design.
- **Marketplace với 10 loại Order khác nhau (digital, physical, subscription, ...):** Có thể leak lên architectural nếu mỗi loại có service riêng.

Hệ quả: bạn không thể học một danh sách “10 quyết định luôn là architectural”. Phải apply framework 3 tiêu chí trong context của mình.

Levels of Knowledge (Knowledge Triangle)

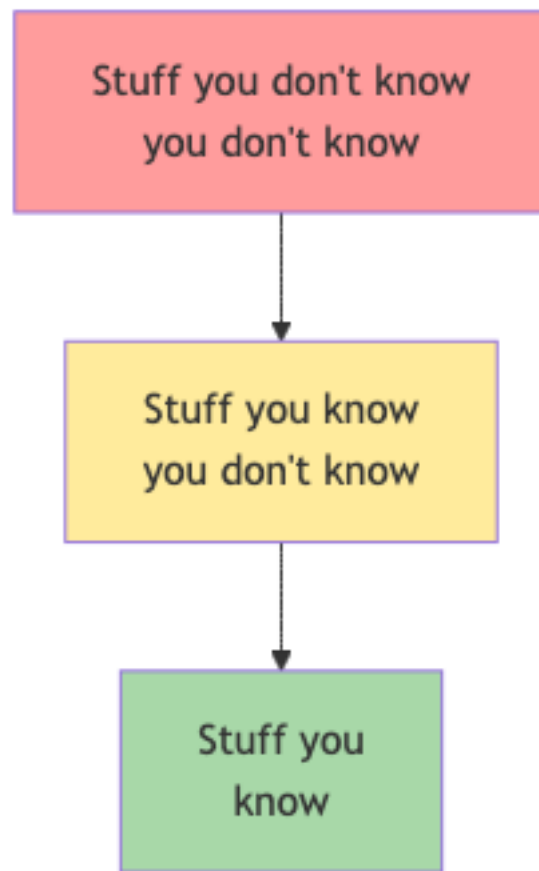
Concept quan trọng cho mọi technologist, đặc biệt architect. Tam giác kiến thức:

Ba mức:

1. Stuff you know (Bạn biết)

Kiến thức bạn đã master: ngôn ngữ, framework, tool, pattern bạn dùng hàng ngày. Vd: developer Java biết Spring, JPA, biết viết unit test.

Đây là cái dùng hàng ngày. Mức này dễ measure (qua certification, code output).



Hình 2: Sơ đồ 7

2. Stuff you know you don’t know (Bạn biết là bạn chưa biết)

Kiến thức bạn *aware* về sự tồn tại nhưng chưa master. Vd: bạn nghe nói về Kafka, biết nó dùng để stream events, nhưng chưa set up production.

Đây là vùng “I can Google this when I need”. Quan trọng: bạn biết tên gọi, biết khi nào nên cân nhắc, biết hỏi ai/đọc gì.

3. Stuff you don’t know you don’t know (Bạn không biết là mình không biết)

Kiến thức bạn *chưa bao giờ nghe đến*. Vd: junior developer chưa biết về CAP theorem, không biết về retry storm, không biết về thundering herd. Khi gặp vấn đề, không biết tên gọi để Google.

Đây là vùng nguy hiểm nhất. Bạn ra quyết định ngây thơ vì không biết có pattern/anti-pattern liên quan.

Vì sao quan trọng cho architect

Đặc thù của architect: **breadth > depth**. Architect không cần master 20 framework, nhưng cần biết tên gọi và high-level ý của 200 concept để:

- Khi vấn đề xảy ra, biết tên gọi để Google sâu hơn.
- Khi thảo luận với expert (vd: DBA, security engineer), hiểu họ nói gì.
- Khi thiết kế hệ mới, biết options thay vì rerolling solution đã có.

□ Architect cần đầu tư vào vùng 2 (stuff you know you don’t know), không cần đào sâu mọi thứ thành vùng 1.

Cách mở rộng vùng 2

- Đọc rộng: 1-2 sách kiến trúc/năm, blog posts hàng tuần (vd: Martin Fowler blog, Microsoft Architecture Center).
- Đi conference / nghe podcast: nghe các architect khác nói về vấn đề họ gặp.
- Pair với expert ở mỗi domain (DBA, security, performance) ít nhất 1 lần để hiểu vocabulary.
- Survey paper / SoK: đọc paper systematization-of-knowledge để biết “ngành đang giải vấn đề gì”.

“Architecture is the stuff that’s hard to Google”

Quote nổi tiếng. Implication: với kiến thức đã có pattern Google được (vd: cách implement REST API, cách viết unit test), không cần architect. Architect cần thiết khi vấn đề *chưa có một câu trả lời chuẩn*, phải lập luận trade-off cho context cụ thể.

Ví dụ:

- “Cách parse JSON trong Python” □ Google được, không cần architect.
- “Có nên tách microservice User và Auth ra hay gộp” □ không Google được câu trả lời đúng, phải cân nhắc context. Đây là việc architect.

Hệ quả thực hành: architect dành thời gian giải các vấn đề *unique to context*. Đừng waste effort optimize những thứ có pattern chuẩn.

Architect cũng phải code

Một xu hướng tai hại: architect “thăng tiến” khỏi việc code, chỉ vẽ diagram và họp. Hậu quả:

- Diagram không khớp với thực tế. Architect không biết constraint hiện tại của codebase.

- Đề xuất giải pháp lý thuyết, không thực dụng.
- Mất uy tín với team: “ông này chỉ nói lý thuyết, có làm bao giờ đâu”.

Bài 3.3 sẽ nói chi tiết hơn. Quy tắc thực hành: architect nên code 20-40% thời gian. Đủ để:

- Hiểu pain hiện tại của codebase.
- Test prototype của đề xuất mới trước khi rollout.
- Pair với developer để mentor và học từ họ.

Sai lầm thường gặp

Sai lầm 1: Coi mọi diagram là architecture

Vẽ một diagram không tự động làm nó “architecture”. Architecture là *các quyết định khó sửa*, diagram chỉ là cách hiển thị một số quyết định.

Một sequence diagram cho một use case là design, không phải architecture. Một deployment diagram cho cluster là architecture vì nó capture quyết định về tổ chức service.

Sai lầm 2: Tranh cãi mọi quyết định ở level architectural

Hiệu quả nghịch. Team mất thời gian tranh cãi việc nhỏ. Heuristic: chỉ escalate lên architectural khi có lý do (3 tiêu chí).

Sai lầm 3: Bỏ qua context khi học pattern

Đọc một bài blog “microservices best practice” rồi áp dụng máy móc cho startup MVP. Pattern luôn có context. Hãy hỏi: “Pattern này được tạo ra cho context nào? Context của tôi có giống không?”.

Sai lầm 4: Quá xa rời code

Đã nói. Architect xa code □ ra quyết định lý thuyết.

Tóm tắt

- **Architecture vs Design**: ranh giới mềm, phụ thuộc context. Dùng framework 3 tiêu chí (bao trùm, ảnh hưởng QA, đắt khi sửa) để phân loại.
- **Cost-of-change** scale theo cấp: implementation (phút), design (giờ), architecture (tháng).
- **Levels of Knowledge**: architect cần mở rộng vùng “know you don’t know” (breadth > depth).
- **“Architecture is the stuff you can’t Google”**: architect giải các vấn đề unique to context.
- **Architect phải code**: 20-40% thời gian, để stay grounded.

Bài tiếp: [Phân tích Trade-off](#), cách lập luận có hệ thống khi mọi quyết định có cost.

3.3 Phân tích Trade-off

Tóm tắt một dòng: Không có “best architecture”, chỉ có “best given priorities”. Job của architect là liệt kê trade-off có hệ thống, không phải pick một option và defend nó. Cùng dạy về cân bằng vai trò architect với hands-on coding.

Câu hỏi nền tảng

“Có nên dùng microservices không?” là câu hỏi sai. Đúng phải là: “Cho project X với priorities Y, microservices đem lại lợi ích Z với cost W. So với monolith thì Z' / W' . Mình ưu tiên gì?”.

Mỗi quyết định architecture đều có:

- **Lợi ích** (benefits), cái nó cải thiện.
- **Chi phí** (costs), cái nó hy sinh hoặc thêm vào.
- **Risk**, điều có thể đi sai.

Architect chuyên nghiệp không “tin” vào style/pattern nào. Họ liệt kê 3 yếu tố trên cho mỗi option, present cho team/stakeholder, hỗ trợ chọn dựa trên *priority hiện tại* của business.

Nếu bạn đến từ Data Science

Trong Data Science, bạn đã quen với trade-off, chỉ là bạn thường gọi nó bằng tên khác. Bạn cân precision và recall. Bạn cân bias và variance. Bạn cân model complexity và overfitting. Software Architecture cũng vậy, nhưng trade-off nằm ở mức hệ thống.

Ví dụ, câu hỏi “có nên deploy model online không?” không thể trả lời bằng câu “online hiện đại hơn”. Bạn phải so các option:

Option	Lợi ích	Giá phải trả
Batch scoring mỗi đêm	Rẻ, đơn giản, dễ audit	Prediction stale, không realtime
Online inference CPU	Fresh hơn, tích hợp API dễ	Cần quản lý latency, autoscaling, uptime
Online inference GPU	Chạy model lớn nhanh hơn	Cost cao, scheduling khó, cold start
Hybrid batch + online rerank	Cân bằng tốt	Logic và monitoring phức tạp hơn

Đây là cùng tư duy với model selection, nhưng đơn vị tối ưu không còn là AUC. Đơn vị tối ưu là latency, freshness, cost, reliability, explainability, privacy và khả năng vận hành.

Vì sao trade-off khó

Vài lý do trade-off thường bị làm sai:

Lý do 1: Lợi ích thường visible, chi phí thì không

Dễ thấy: “Microservices scale từng phần độc lập!”, “Event-driven async cho high throughput!”. Khó thấy: complexity của distributed tracing, network failures, eventual consistency bugs.

Architect ngây thơ chỉ list lợi ích $\square$ quyết định sai.

Lý do 2: Chi phí phân tán theo thời gian

Chọn microservices thấy đau ngay tuần 1 (setup phức tạp), nhưng cũng đau ở tháng 24 (operational overhead). Phần lớn cost ở tháng 24, và architect ngây thơ không cân nhắc khi quyết định ở tuần 1.

Lý do 3: Stakeholder khác nhau ưu tiên khác nhau

CTO ưu tiên scalability (long-term). PM ưu tiên time-to-market (short-term). DBA ưu tiên data consistency (correctness). Architect là người ghép các view này lại, không phải chọn 1 view.

Lý do 4: Khó định lượng

“Maintainability” khó đo bằng số. So sánh “monolith maintainable hơn microservices vì simpler” với “microservices maintainable hơn monolith vì module boundary cứng”, không có metric nào quyết định.

Framework ATAM-lite

ATAM (Architecture Tradeoff Analysis Method) là phương pháp formal của SEI Carnegie Mellon. Đầy đủ ATAM tốn cả tuần workshop. Bản gọn ATAM-lite cho việc hàng ngày:

Bước 1: Liệt kê options (3-5 cái)

Đừng so 2 options, nó dễ trở thành “tôi vs anh” và mất objectivity. List 3-5 options bao gồm “do nothing” và “naive solution” làm baseline.

Ví dụ cho câu hỏi “Nên dùng cache cho hệ tìm kiếm không?”:

- Option A: Không cache, mọi request đi đến database.
- Option B: Cache in-memory per-instance (vd: LRU).
- Option C: Cache distributed (Redis).
- Option D: CDN cache cho response.

Bước 2: Liệt kê quality attributes liên quan

Cho mỗi option, xem ảnh hưởng tới các QA. QA phổ biến:

- Performance (latency, throughput).
- Scalability.
- Availability.
- Consistency (data correctness).
- Maintainability.
- Cost (infrastructure, dev time).
- Security.
- Observability.

Vẽ bảng:

QA	A (no cache)	B (local)	C (Redis)	D (CDN)
Latency	High (DB query)	Low	Low	Lowest
Throughput	Low	High	High	Highest
Consistency	Strong	Eventual (TTL)	Eventual	Eventual

QA	A (no cache)	B (local)	C (Redis)	D (CDN)
Cost infra	Low	Low	Medium	Low
Cost dev	Low	Medium	Medium	High (CDN config)
Maintainability	High	High	Medium (Redis ops)	Medium
Observability	Easy	Hard (per-inst)	Medium (centralized)	Hard

Bước 3: Identify trade-off

Trade-off = QA cải thiện ở option này, xấu đi ở option khác. Vd:

- Option B trade Consistency lấy Throughput.
- Option C thêm Cost infra để có Throughput + observability tốt hơn B.
- Option D có Latency tốt nhất nhưng phức tạp setup nhất.

Bước 4: Map vào priorities

Hỏi business/team:

- “Latency p99 quan trọng cỡ nào? 200ms acceptable?”
- “Stale data 30s có ổn không cho use case này?”
- “Team có experience operate Redis không?”

Câu trả lời dẫn tới decision. Ví dụ:

- “Tolerate 30s stale + cần p99 < 100ms + team không có Redis ops” □ Option D (CDN).
- “Cần real-time + chấp nhận latency cao” □ Option A (no cache).
- “Cần balance + có ops team” □ Option C (Redis).

Bước 5: Document quyết định

Viết ADR (Architecture Decision Record). Format chuẩn:

```
# ADR-007: Caching strategy for search service
```

```
## Context
```

- Search latency p99 đang là 800ms, target 200ms.
- 80% queries lặp lại trong 5 phút.
- Stale data 1-2 phút acceptable cho UX.

```
## Options considered
```

1. No cache (status quo)
2. Local LRU cache
3. Redis distributed cache
4. CDN cache

```
## Decision
```

Chọn option 3 (Redis distributed cache).

```
## Rationale
```

- Latency: dự kiến giảm xuống 50-100ms (qua benchmark prototype).

- Throughput: tăng 5-10x.
- Observability: centralized Redis có Prometheus exporter sẵn.
- Team đã có experience Redis ops từ project khác.
- Cost: ~\$200/tháng cho Redis cluster, acceptable.

Consequences

- Phải code cache invalidation logic.
- Phải monitor Redis cluster health.
- Eventual consistency: stale 30-120s tùy TTL.

Status: Accepted, 2026-01-15

ADR là nơi lưu trade-off analysis. Future maintainer sẽ hiểu *vì sao* quyết định này, không chỉ *cái gì* được quyết.

Sai lầm “tối ưu một chiều”

Sai lầm phổ biến: architect tập trung vào 1 QA và bỏ qua các QA khác.

Ví dụ thật

Một startup hype microservices vì “scalability”. Quyết định: tách monolith 50 services. Một năm sau:

- Scalability: đạt (mỗi service scale riêng).
- Maintainability: TỆ HƠN (distributed tracing thiếu, debug cross-service mất nhiều giờ).
- Time-to-market: TỆ HƠN (feature mới touch 5-10 services).
- Cost ops: TĂNG 5x (Kubernetes, service mesh, monitoring stack).
- Team: BURN-OUT (on-call lan rộng, không ai own end-to-end).

Net effect: âm. Vì chỉ tối ưu scalability mà bỏ qua các QA khác.

Cách phòng tránh

Trước mỗi quyết định lớn, hỏi: “Tôi đang tối ưu QA nào? Tôi đang hy sinh QA nào? Hy sinh đó có acceptable không?”.

Nếu không answer được “đang hy sinh gì” □ bạn chưa thấy trade-off □ analysis chưa đủ.

Cân bằng Architect và Hands-on Coding

Một dimension trade-off ít người nói: thời gian architect dành cho diagram/document vs cho coding.

Hai cực sai

Cực 1: Architect 100% non-coding (“Ivory Tower Architect”):

- Vẽ diagram đẹp.
- Đề xuất giải pháp lý thuyết.
- Mất uy tín với team (“ông này không biết code base của mình”).
- Quyết định disconnect với reality.

Cực 2: Architect 100% coding (“Senior Developer in disguise”):

- Code tốt nhưng không đầu tư cross-cutting work.

- Không document.
- Không mentor team.
- Quyết định ad-hoc.

Sweet spot

Empirical: 20-40% thời gian architect dành cho coding. Code:

- Prototype cho đề xuất mới (POC).
- Review PR có touch architecture.
- Pair với developer mid-level để mentor.
- Build common library / framework dùng across team.

60-80% còn lại:

- Diagram + documentation (ADR).
- Cross-team meeting.
- Code review (không write).
- Technical strategy long-term.
- Mentorship 1-on-1.

Tỷ lệ tùy team size:

- Startup 5 người: architect có thể code 60-70% (vì codebase nhỏ, tech debt phải clear).
- Enterprise 100 engineers: architect có thể code 10-20% (vì cross-cutting work nhiều).

Dấu hiệu architect đã xa rời code

- Lần cuối merge code > 6 tháng.
- Không biết build command của project mình.
- Không biết test pass-rate của repo chính.
- PR review chỉ “LGTM” không có substantial comment.

Nếu bạn (hoặc senior architect bạn quen) thấy 2+ dấu hiệu, cần realign ngay.

Trade-off đặc biệt: “Right-sizing” architecture

Đừng over-engineer (architecture quá phức tạp so với problem), cũng đừng under-engineer (architecture không đỡ được growth).

Quy tắc thực hành: “Architecture should match team size + 1 stage”

- 1-5 people team: monolith, 1 database, deploy as one. Đừng bao giờ microservices ở stage này.
- 5-20 people: monolith tổ chức tốt thành modules, có thể tách 1-2 service nếu có lý do.
- 20-50 people: modular monolith hoặc service-based architecture. 4-12 services.
- 50-200 people: microservices. 20-100 services.
- 200 people: microservices + platform team build internal tools.

Đi quá nhanh (5 people làm microservices) = over-engineering. Đi quá chậm (200 people 1 monolith) = bottleneck.

Sai lầm thường gặp

Sai lầm 1: “Hype-driven architecture”

Đọc bài blog hot, áp dụng ngay. Không xét context.

Sai lầm 2: “Resume-driven architecture”

Chọn tech để CV trông ngầu. Hệ quả: team dùng K8s+Istio+gRPC cho app 100 user.

Sai lầm 3: “Last project bias”

Architect vừa làm microservices ở công ty cũ thành công □ đề xuất microservices cho công ty mới mà không kiểm tra fit.

Sai lầm 4: “Default to complex”

Khi không chắc, chọn solution phức tạp “for future flexibility”. Sai. Default nên là *simplest thing that works*, phức tạp khi cần justify được.

Tóm tắt

- **Trade-off analysis** = liệt kê options, map QA, identify trade-off, match priority.
- **ATAM-lite**: 5 bước, đủ cho việc hàng ngày.
- **Document qua ADR**: future maintainer hiểu vì sao.
- **Sai lầm “tối ưu một chiều”**: luôn hỏi “đang hy sinh gì”.
- **Architect coding 20-40%**: stay grounded.
- **Right-size architecture**: match team size + 1 stage.

Bài tiếp: [Modularity](#), scale cohesion-coupling lên mức hệ thống.

3.4 Modularity

Tóm tắt một dòng: Modularity ở mức kiến trúc là chia hệ thống thành các đơn vị có high cohesion + low coupling. Chia *vertical* (theo feature/domain) thường tốt hơn *horizontal* (theo layer), và sai modularity dẫn tới các anti-pattern tệ nhất ngành: big ball of mud và distributed monolith.

Vì sao quan trọng

Trong Bài 2.2, bạn đã học cohesion-coupling ở mức class/module. Phần này áp dụng cùng concept ở quy mô lớn hơn: chia hệ thống thành sub-system, service, hoặc bounded context.

Modularity sai ở mức hệ thống dẫn tới hai anti-pattern tệ nhất:

Big Ball of Mud

Codebase không có structure rõ ràng. Mọi class depend mọi class. Sửa A có thể vỡ B, C, D ở nơi không lường được. Mỗi feature mới mất nhiều ngày. Onboarding new dev mất nhiều tháng.

Đây là kết quả của *thiếu modularity*: không có module boundary rõ.

Distributed Monolith

Có vẻ là microservices (mỗi service deploy riêng), nhưng services depend chặt chẽ vào nhau. Đổi 1 service phải đổi 5 services khác. Deploy phải coordinate. Cuối cùng có *cost* của microservices (network, complexity) mà không có *lợi ích* (independent scale + deploy).

Đây là kết quả của *modularity sai chỗ*: tách service theo layer thay vì theo domain.

Cả hai anti-pattern này phòng được nếu hiểu modularity đúng.

Nếu bạn đến từ Data Science

Một DS project cũng có modularity, dù đôi khi nó không được gọi như vậy. Hãy nhìn một ML system production đơn giản:

```
ingestion -> validation -> feature engineering -> training -> registry -> serving -> monitoring
```

Nếu tất cả nằm trong một notebook hoặc một script `train.py`, bạn có Big Ball of Mud phiên bản Data Science. Nếu bạn tách thành 20 microservices nhưng mỗi thay đổi feature phải sửa training service, serving service, monitoring service và dashboard cùng lúc, bạn có Distributed Monolith phiên bản ML platform.

Boundary tự nhiên của ML/data system thường không phải “file Python nào gọi file nào”, mà là trách nhiệm thay đổi:

- Data ingestion thay đổi khi nguồn dữ liệu thay đổi.
- Feature engineering thay đổi khi định nghĩa feature thay đổi.
- Training thay đổi khi thuật toán hoặc objective thay đổi.
- Model registry thay đổi khi lifecycle/versioning thay đổi.
- Serving thay đổi khi latency/scale/deployment thay đổi.
- Monitoring thay đổi khi cần detect drift, data quality hoặc performance.

Chia module tốt nghĩa là mỗi thay đổi trên không kéo cả hệ thống đi theo.

Nguyên tắc: High cohesion at module boundary

Khi chia hệ, mỗi module nên thoả:

1. **Internal high cohesion**: mọi thứ trong module cùng phục vụ một mục đích.
2. **External low coupling**: module giao tiếp với module khác qua minimal, well-defined interface.
3. **Domain alignment**: module map vào một “thing” có ý nghĩa với business, không phải vào technical concern.

Tiêu chí 3 là cốt lõi của khái niệm **bounded context** trong Domain-Driven Design. Module nên reflect business, không reflect tech stack.

Vertical vs Horizontal Slicing

Một quyết định lớn khi modularize: chia theo *layer technical* hay theo *feature business*.

Horizontal Slicing (theo layer)

Chia hệ thành tầng:

Presentation Layer (UI/API)
Business Logic Layer
Data Access Layer
Database

Mỗi tầng “own” by một team:

- UI team own Presentation.
- Backend team own Business.
- Data team own DAL.

Nghe có vẻ hợp lý. Nhưng có vấn đề:

Vấn đề 1: Feature đung mọi tầng Một feature “đặt hàng” cần thay đổi UI (form), Business (validate, calculate), DAL (save order). Mỗi feature = coordinate 3 team. Delay rất nhiều.

Vấn đề 2: Conway’s law xung đột Org chart (theo team) khớp với layer. Nhưng business value flow *xuyên qua* tầng. Conflict.

Vấn đề 3: Low cohesion mức module UI module chứa code của 100 feature khác nhau (vì UI cho mọi feature). Cohesion thấp.

Vertical Slicing (theo feature/domain)

Chia hệ thành “domain”:

Orders	Payments	Users
UI logic	UI logic	UI logic
Business	Business	Business
Data	Data	Data

Mỗi domain “own” by một team. Team Orders own toàn bộ UI + Business + Data của Orders.

Lợi ích:

- **Feature đúng 1 domain:** team Orders thêm feature mới chỉ touch code Orders. Fast.
- **High cohesion:** code trong Orders tập trung vào orders.
- **Conway’s law align:** org chart ánh xạ business domain.
- **Independent deploy:** thay đổi Orders không ảnh hưởng Payments.

Khi nào dùng cái nào

Aspect	Horizontal	Vertical
Team org	Theo skill (frontend, backend)	Theo domain
Best for	Small team, MVP, có technical specialty cao	Lớn, đa team, complex domain
Anti-pattern	Big ball of mud horizontal	Big ball of mud vertical
Mở rộng	Khó (mỗi layer scale riêng không help feature)	Dễ (mỗi domain scale riêng)

Heuristic: **vertical slicing là default tốt cho hệ trung-lớn**. Horizontal slicing chỉ tốt khi team < 10 và domain đơn giản.

Bounded Context (từ DDD)

Khái niệm cốt lõi của DDD, áp dụng được mà không cần học full DDD. Bounded context = phạm vi mà một model có nghĩa nhất quán.

Ví dụ: từ “Customer” có nghĩa khác trong các context khác nhau:

- **Sales context:** Customer = potential buyer, có lead score, sales rep assigned.
- **Billing context:** Customer = entity được charge, có payment method, billing address.
- **Support context:** Customer = user có ticket history, satisfaction score.

Sai lầm: tạo một class Customer god class chứa mọi attribute từ mọi context. Class này phình ra 50+ field, ai cũng phải hiểu cả 50 attribute để chạm vào.

Đúng: mỗi bounded context có Customer model riêng. Mapping (translation) qua boundary khi cần.

Mỗi bounded context = một module/service. Đây là cách phổ biến nhất để identify boundary trong vertical slicing.

Anti-patterns chi tiết

Anti-pattern 1: Big Ball of Mud (BBoM)

Code không có structure. Mọi class import mọi class. Module name không phản ánh content.

Triệu chứng:

- Mở random file, không đoán được nó làm gì từ tên.
- Sửa một method có thể vỡ ở vài chục nơi không lường trước.
- Test coverage thấp vì không thể test isolated.
- Onboarding mất nhiều tháng.

Nguyên nhân: tích lũy tech debt mà không refactor; thiếu architectural guidance; team turnover cao mà không document.

Cách thoát: lựa chọn vài bounded context rõ ràng nhất □ refactor một-một thành module rõ. Đừng cố fix toàn bộ một lần.

Anti-pattern 2: Distributed Monolith

Tách thành nhiều service nhưng services depend chặt vào nhau.

Triệu chứng:

- Deploy phải coordinate (deploy A xong mới được deploy B).
- Database shared giữa các service.
- Service A gọi B gọi C gọi A (cyclic dependency).
- Một service down □ toàn hệ thống vỡ.

Nguyên nhân: tách service theo *layer* thay vì *domain* (horizontal slicing nhưng deploy riêng); shared database; lack of bounded context.

Cách thoát: đảo về monolith well-organized (modular monolith), sau đó tách lại theo bounded context đúng.

Anti-pattern 3: Death Star

Service hub central depend vào (hoặc bị depend bởi) hầu hết các service khác.

Hub trở thành single point of failure và bottleneck. Đổ hub đau cả hệ.

Cách thoát: phân tích phụ thuộc, tách hub thành nhiều service nhỏ hơn theo bounded context.

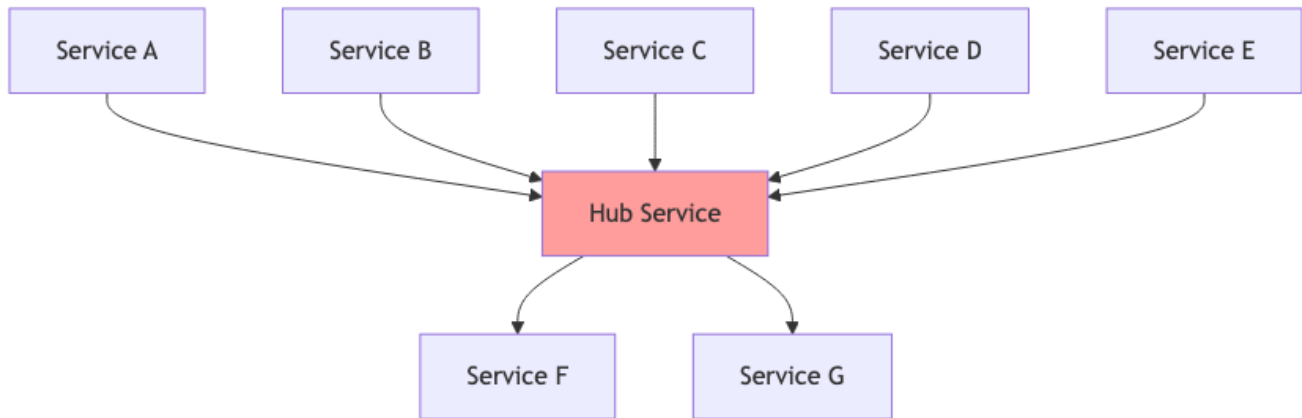
Modularity ở các context khác nhau

Trong monolith

Tách thành **modules / packages**. Mỗi module có boundary rõ qua interface công khai.

Ví dụ Java/Spring:

```
com.company.app/
├─ orders/           # Module Orders
|  └─ api/           # Public API (exposed)
|  └─ domain/        # Internal models
|  └─ infrastructure/ # DB, etc.
```



Hình 3: Sơ đồ 8

```

|   └─ OrdersFacade.java # External entry point
└─ payments/              # Module Payments
└─ users/                 # Module Users
  
```

Quy tắc: code trong orders/ không import trực tiếp từ payments/domain/*, chỉ từ payments/api/*. Build tool có thể enforce qua module-info.java (Java 9+).

Trong microservices

Mỗi service = một module. Boundary = network call. Quy tắc same:

- Service Orders không truy cập database của Payments.
- Service Orders không gọi internal API của Payments, chỉ gọi public API/event.
- Mỗi service có own database (database-per-service pattern).

Trong mobile apps

Chia thành **feature modules**. Vd Android Gradle modules:

```

:app                # entry point
:feature:orders     # Orders feature
:feature:payments   # Payments feature
:core:network       # Shared infrastructure
:core:ui            # Shared UI components
  
```

Build tool prevent feature modules depend lẫn nhau. Shared code go vào :core:.*.

Trong frontend SPA

Module federation, micro-frontends. Mỗi vertical slice = một bundle riêng. Vd Module Federation của Webpack 5.

Heuristic identify module boundary

Khi nhìn vào hệ phức tạp, làm sao biết chia ở đâu? Vài heuristic:

Heuristic 1: Theo “Two-Pizza Team” của Amazon

Một module/service nên đủ nhỏ để 2 pizza (6-8 người) own end-to-end. Lớn hơn = cần tách. Nhỏ hơn = có thể gộp.

Heuristic 2: Theo change rate

Code thay đổi cùng nhau nên ở cùng module. Code thay đổi độc lập nên tách module.

Đo: phân tích git log. Files được commit cùng nhau thường xuyên (>40% commits) □ cohesion cao □ cùng module. Files hiếm khi cùng commit □ coupling thấp □ có thể tách.

Tool: codescene.com tự động hoá analysis này.

Heuristic 3: Theo data ownership

Nếu hai concept share data heavily (cùng đọc/ghi cùng tables) □ có thể ở cùng module.

Nếu hai concept chỉ trao đổi data qua event/API rời rạc □ có thể tách module.

Heuristic 4: Theo bounded context

Map từ DDD. Identify ubiquitous language của mỗi business domain. Cùng từ vựng □ cùng bounded context □ cùng module.

Cẩn thận: Modularity có cost

Tách module thêm:

- Boilerplate (interface, facade).
- Build complexity.
- Cross-module testing.
- Runtime overhead (nếu cross-network).

Nguyên tắc: tách khi pain xuất hiện. Đừng modularize prophylactically.

Counter-example: startup MVP với 3 developer chia thành 8 modules, over-engineering. 1 monolith file 5000 dòng OK cho MVP, tách khi grow.

Tóm tắt

- **Modularity ở mức hệ thống**: cohesion-coupling scale lên.
- **Vertical (theo domain) thường tốt hơn horizontal (theo layer)** cho team trung-lớn.
- **Bounded context (DDD)** là cách identify domain boundary đúng.
- **Anti-pattern**: Big Ball of Mud, Distributed Monolith, Death Star.
- **Heuristics**: two-pizza team, change rate, data ownership, bounded context.
- **Cẩn thận**: tách khi pain xuất hiện, không phòng ngừa.

Tổng kết Phần 3

Hết Phần 3. Tóm tắt:

Bài	Insight chính
3.2 Architecture vs Design	Ranh giới mềm theo context; framework 3 tiêu chí; Levels of Knowledge
3.3 Trade-off	Mọi quyết định có cost; ATAM-lite; tránh tối ưu một chiều
3.4 Modularity	Vertical > Horizontal; bounded context; tránh BBoM/distributed monolith

Tư duy architect đã được set up. Phần tiếp [Quality Attributes](#) sẽ bổ sung *cái cần tối ưu* cho trade-off analysis.